

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	B.Tech Computer Science and Biosciences	Date:	1.8.2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

ANSWER

1. Identification of the error / bug

An **8-bit signed 2's complement** number can represent values only from **-128 to +127**. Both operands (+120 and +50) are valid individually, but their mathematical sum is **+170**, which is outside this range. The stored 8-bit pattern therefore wraps around and appears as a negative number. The bug is not ordinary carry out; it is **signed overflow**.

2. Binary debugging analysis

Item	Sign bit	8-bit representation	Meaning
+120	0	01111000	Positive operand
+50	0	00110010	Positive operand
8-bit sum	1	10101010	Interpreted as -86
True 9-bit sum	0	010101010	+170

Binary addition: $01111000 + 00110010 = 10101010$ when only eight result bits are retained. The two inputs have **sign bit 0**, but the result has **sign bit 1**. Hence two positive numbers produced a negative result, which proves overflow. The same condition is detected in hardware when the **carry into the sign bit** differs from the **carry out of the sign bit**.

Overflow flag equation: **$V = \text{NOT}(A7 \text{ XOR } B7) \text{ AND } (A7 \text{ XOR } S7)$** , where A7 and B7 are the operand sign bits and S7 is the result sign bit. For this addition, A7 = 0, B7 = 0, and S7 = 1, so **$V = 1$** .

3. Corrected solution

The adder should calculate the normal 8-bit sum and the **overflow flag** together. If $V = 0$, the result is safe. If $V = 1$, the system must not treat the wrapped pattern as the correct signed answer. It can either signal an **overflow exception**, set a **status flag**, use **saturation**, or repeat the addition in a **wider format**.

Handling method	Action for +120 + +50	Suitable use
Widening	Extend the operand signs to 9 or more bits; store +170	Exact arithmetic
Saturation	Clamp result to +127 and set overflow flag	Audio/control systems
Exception/status	Reject 10101010 as a valid signed result	Processors and safety checks

Corrected control logic: first add the operands, then evaluate the **sign bit overflow equation**. When overflow is detected, the program should display '**Overflow: exact sum requires more than 8 bits**' and compute the answer using a **wider integer type**.

4. Optimization applied

Overflow detection needs only a few **XOR, NOT, and AND gates** and can operate **in parallel** with the final addition. This is faster and safer than converting every result to decimal or checking all bits after the operation. A **wider adder** is used only when exact values outside the range must be retained.

5. Testing and validation

Test	Mathematical result	8-bit result	Overflow?	Expected handling
+120 + +50	+170	10101010 (-86)	Yes	Flag; widen or saturate
+100 + +20	+120	01111000 (+120)	No	Accept result
-100 + -40	-140	01110100 (+116)	Yes	Flag; widen or saturate
-100 + +50	-50	11001110 (-50)	No	Accept result

Result: Overflow is correctly detected for +120 + +50; the wrapped value -86 is rejected, and the exact widened result is +170.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

ANSWER

1. Identification of the performance bug

In a **ripple carry adder (RCA)**, each full adder stage must wait for the carry from the previous stage. The sum bit at a high position cannot become final until the carry has rippled through every lower bit. The root cause of the **delay in the real time system** is therefore a long **serial carry propagation path**, not an incorrect arithmetic result.

2. Debugging analysis of carry propagation

For bit i , define the generate term $G_i = A_i \text{ AND } B_i$ and the propagate term $P_i = A_i \text{ XOR } B_i$. The next carry is $C_{i+1} = G_i \text{ OR } (P_i \text{ AND } C_i)$. An RCA evaluates this relation one stage at a time: C_1 depends on C_0 , C_2 depends on C_1 , and so on. For an n -bit adder, the worst case carry delay grows approximately **in proportion to n** .

4-bit example	A	B	Carry behaviour
Input	1111	0001	$C_0 = 0$
Bit 0	1	1	Generates C_1
Bits 1 to 3	1	0	Each stage propagates the carry
Final	-	-	C_4 appears only after the full chain

Timing probes at **C_1 , C_2 , C_3 , and C_4** would show the transitions arriving sequentially. This confirms that the **critical path** passes through four carry gates for a 4-bit RCA and through many more gates in a wider processor adder.

3. Optimized solution: carry look ahead adder

A **carry look ahead adder (CLA)** expands the carry equations so that the carries are calculated from the original inputs **in parallel** instead of waiting for the preceding stage.

Carry	Look ahead equation
C_1	$G_0 \text{ OR } P_0 C_0$
C_2	$G_1 \text{ OR } P_1 G_0 \text{ OR } P_1 P_0 C_0$
C_3	$G_2 \text{ OR } P_2 G_1 \text{ OR } P_2 P_1 G_0 \text{ OR } P_2 P_1 P_0 C_0$
C_4	$G_3 \text{ OR } P_3 G_2 \text{ OR } P_3 P_2 G_1 \text{ OR } P_3 P_2 P_1 G_0 \text{ OR } P_3 P_2 P_1 P_0 C_0$

Once P_i and G_i are available, the look ahead network produces the carry signals with a small, bounded number of logic levels. The sum is then $S_i = P_i \text{ XOR } C_i$. For large word sizes, 4-bit or 8-bit CLA blocks can be combined **hierarchically** using **group propagate** and **group generate** signals.

4. Performance improvement

Feature	Ripple carry adder	Carry look ahead adder
Carry evaluation	Serial, bit after bit	Parallel look ahead network
Worst case delay	Approximately $O(n)$	Approximately $O(\log n)$ with hierarchy
Hardware area	Low	Higher due to extra logic
Best use	Small, economical circuits	Fast arithmetic in real time systems

The CLA uses more gates and wiring, but it removes the long ripple path. In a real time processor, the **reduced critical path delay** permits a **shorter clock period** and faster arithmetic execution.

5. Testing and validation

Input A	Input B	Cin	Expected sum	Purpose
1111	0001	0	1 0000	Worst case carry propagation
0101	0010	0	0 0111	No long carry chain
1010	0101	1	1 0000	Checks the input carry path
0000	0000	0	0 0000	Zero case

Both adders must produce **identical sums** for all input combinations. The validation is successful only if the CLA matches the RCA's arithmetic result while timing analysis shows that C4 and the most significant sum bit **settle earlier**.

Result: The bottleneck is the RCA's serial carry chain. A hierarchical CLA computes carries concurrently and reduces the delay while preserving the correct sum.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

ANSWER

1. Identification of possible errors

Incorrect results for negative operands usually come from one or more implementation errors: reversing the **Q0,Q-1 pair**, using a **logical right shift**, failing to **extend the sign** of the multiplicand, shifting A and Q separately, or executing the wrong number of cycles. The most common root cause is that a logical shift inserts 0 into the sign position and destroys a **negative partial product**.

Bit pair Q0 Q-1	Required operation on A
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

2. Step by step debugging: (-5) x (+3)

Use 4-bit operands **M = 1011 (-5)**, **Q = 0011 (+3)**, accumulator A = 0000, and Q-1 = 0. After each pair decision, perform one **arithmetic right shift** on the combined register **[A,Q,Q-1]**.

Cycle	Q0 Q-1	Operation before shift	A after operation	After arithmetic shift: A Q Q-1
Initial	-	-	0000	0000 0011 0
1	10	$A = A - M$	0101	0010 1001 1
2	11	No operation	0010	0001 0100 1
3	01	$A = A + M$	1100	1110 0010 0
4	00	No operation	1110	1111 0001 0

After four cycles, the product is the combined A,Q value **11110001**. Interpreting this 8-bit 2's complement number gives **-15**, which is the correct value of $(-5) \times (+3)$. If cycle 3 used a logical shift, A would begin with 0 instead of **retaining 1**, and the final result would be incorrect.

3. Corrected Booth workflow

1. Store M and Q in **n-bit 2's complement** form; use an accumulator wide enough to preserve the sign.
2. Initialize **A = 0** and **Q-1 = 0**.
3. Inspect the pair in the exact order **Q0,Q-1** and apply the 00/01/10/11 table.
4. **Extend the sign of M** before every addition or subtraction.
5. **Arithmetic right shift** the entire [A,Q,Q-1] register so that A's sign bit is copied.
6. Repeat **exactly n times** and read the final **2n-bit product** from [A,Q].

Special boundary correction: the **most negative n-bit value** cannot be negated in the same n bits. For example, +8 cannot be represented in 4-bit signed form when **M = -8**. Using an **(n+1)-bit accumulator** and an (n+1)-bit **sign extended multiplicand** prevents this overflow during $A - M$.

4. Optimization applied

Booth encoding **reduces the number of additions** for long runs of 1s in the multiplier. A run such as 0011110 is handled through **transitions at its boundaries** instead of one addition for every 1. The corrected design keeps **one combined shift register**, one adder/subtractor, and a **fixed controller with n cycles**, which improves control simplicity and prevents inconsistent separate shifts.

5. Testing and validation

Multiplicand	Multiplier	Expected 8-bit product	Decimal result	Purpose
-5	+3	11110001	-15	Negative x positive
-5	-3	00001111	+15	Negative x negative
+7	-2	11110010	-14	Positive x negative
-8	+1	11111000	-8	Minimum 4-bit value
-8	-1	00001000	+8	Negation boundary

Result: Correct pair decoding, sign extension, a combined arithmetic shift, and an extra accumulator bit produce the correct signed product for ordinary and boundary negative operands.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

ANSWER

1. Identification of the inefficiency

In **restoring division**, each cycle shifts the partial remainder, subtracts the divisor, and checks the sign. Whenever the remainder becomes negative, the hardware **adds the divisor back** immediately to restore the previous value. That restoration is an **extra arithmetic operation** in every unsuccessful subtraction, so the same partial remainder is **modified twice** in one iteration.

2. Debugging the restoring workflow

Stage	Restoring action	Why delay occurs
1	Left shift [A,Q]	Prepares next quotient bit
2	$A = A - M$	Trial subtraction
3	If $A < 0$, set $Q_0 = 0$	Sign decision
4	If $A < 0$, $A = A + M$	Immediate restoration adds another adder delay
5	Else set $Q_0 = 1$	Successful subtraction retained

A trace counter placed on the adder/subtractor would show that negative trial remainders cause **two operations in a cycle: subtract M** and then **add M**. This repeated restoration is the **performance bottleneck**.

3. Optimized nonrestoring division

Nonrestoring division does not immediately undo a negative result. At the next cycle, the sign of the current partial remainder selects the opposite operation: **subtract the divisor** when A is zero or positive, or **add the divisor** when A is negative. Only **one final correction $A = A + M$** is required if the last remainder is negative.

1. Initialize **$A = 0$** ; load **dividend** into Q and **divisor** into M.
2. For each of **n quotient bits**, shift the combined [A,Q] register left.
3. If A was zero or positive before the shift, perform **$A = A - M$** ; otherwise perform **$A = A + M$** .
4. Set **$Q_0 = 1$** when the new A is zero or positive; otherwise set **$Q_0 = 0$** .
5. After n cycles, if A is negative, perform **one final $A = A + M$** .

4. Worked trace: 13 / 3

Use **Q = 1101 (13)**, **M = 00011 (3)**, and a 5-bit signed **A = 00000**. The four quotient cycles are shown below.

Cycle	Previous sign of A	Operation after shift	A after operation	Q after Q0
1	Zero or positive	$A = A - M$	11110 (-2)	1010
2	Negative	$A = A + M$	00000 (0)	0101
3	Zero or positive	$A = A - M$	11101 (-3)	1010
4	Negative	$A = A + M$	11110 (-2)	0100
Final	Negative	$A = A + M$	00001 (1)	0100 (4)

The final quotient is **Q = 0100 = 4** and the final remainder is **A = 00001 = 1**. Therefore, **13 = (3 x 4) + 1**, which validates the result.

5. Performance comparison and optimization

Feature	Restoring division	Nonrestoring division
Trial failure	Subtract then restore	Keep negative remainder
Arithmetic per cycle	May require two operations	One add or subtract
Final correction	Not normally required	At most one addition
Execution	Longer critical path in failed trials	More regular and faster control

Both methods still use **n iterations**, but nonrestoring division removes repeated restoration in the same cycle. It creates a regular **datapath with one operation per cycle**, **reduces adder activity**, and simplifies timing in the embedded divider.

6. Testing and validation

Dividend	Divisor	Expected quotient	Expected remainder	Validation
13	3	4	1	General case
15	3	5	0	Exact division
7	8	0	7	Dividend < divisor
0	5	0	0	Zero dividend
Any	0	Undefined	Undefined	Must raise a division by zero error

Result: The repeated restoration step is removed. Nonrestoring division returns quotient 4 and remainder 1 for 13 / 3 with one arithmetic action per main cycle and at most one final correction.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

ANSWER

1. Identification of the numerical error

IEEE 754 single precision uses **1 sign bit**, **8 exponent bits**, and **23 stored fraction bits**. With the hidden leading 1, it provides about **24 bits of significant precision**, or roughly **7 decimal digits**. When a very small value is added to a much larger value, the smaller significand may be shifted beyond these available bits and be rounded away. This is **loss of significance** caused by **finite precision**, not necessarily a faulty floating point unit.

2. Debugging exponent alignment, normalization, and rounding

Consider the single precision operation **100,000,000 + 1**. The larger number is close to **2^{26}** . At this magnitude, adjacent single precision values are spaced by **$2^{(26-23)} = 8$** . Therefore, 1 is smaller than one **unit in the last place (ULP)** and cannot be represented as a change to 100,000,000.

Debugging stage	What happens	Observed problem
Exponent alignment	The small significand is shifted right by the exponent difference	Its useful bits move beyond the 24-bit significand
Significand addition	The aligned significands are added	The small contribution may already be only guard/round/sticky bits
Normalization	The result is shifted to the form $1.x \times 2^e$	Another shift may discard a lower order bit
Rounding	Round to nearest, ties to even selects a representable value	The result may remain exactly 100,000,000

Thus, $\text{float}(100,000,000) + \text{float}(1)$ rounds back to 100,000,000, although the mathematical answer is **100,000,001**. A second diagnostic case is **$1.0 + 2^{-25}$** . The ULP at 1.0 is **2^{-23}** , so 2^{-25} is **below half an ULP** and also rounds away under **round to nearest even**.

3. Corrected and optimized solutions

Technique	Correction	Benefit / limitation
Double precision	Use a 53-bit significand for accumulation	$100,000,000 + 1$ is represented accurately; uses more storage/bandwidth
Ascending magnitude summation	Add values of similar magnitude first	Prevents small terms from being lost too early
Pairwise summation	Sum balanced groups and combine group totals	Error grows more slowly than simple sequential summation
Kahan compensation	Track lower order bits lost in each addition	Greatly improves repeated sums with small extra work
Scaling	Rescale data before arithmetic and scale back later	Reduces extreme exponent differences when valid for the problem

For this application, the simplest reliable correction is to store the operands and accumulator as **double precision**. If a large series of values with mixed magnitudes must be summed, combine **double precision** with **pairwise** or **Kahan summation**.

4. Kahan compensated summation logic

Step	Operation	Purpose
1	sum = 0; compensation = 0	Initialize
2	adjusted = value - compensation	Restore the previously lost lower order part
3	temporary = sum + adjusted	Perform the current addition
4	compensation = (temporary - sum) - adjusted	Measure the rounding part that was lost
5	sum = temporary	Continue with corrected state

This adjustment does not create unlimited precision, but it **preserves many small contributions** that ordinary sequential summation would discard. **Guard, round, and sticky bits** should also be retained inside the hardware datapath until the **final IEEE 754 rounding step**.

5. Testing and validation

Operation	Single precision observation	Improved method	Expected improved result
100,000,000 + 1	100,000,000	Double precision	100,000,001
$1.0 + 2^{-23}$	Next float above 1.0	Single is sufficient	Approximately 1.000000119
$1.0 + 2^{-25}$	1.0	Double precision	1.0000000298023224
100,000,000 + many 1s	Many increments may disappear	Kahan or pairwise double sum	Close to mathematical total

Validation should compare the result against a **reference with greater precision** and report both **absolute error** $|\text{computed} - \text{reference}|$ and **relative error**. The test is successful when the optimized method reduces these errors without producing overflow, underflow, **NaN**, or **infinity** unexpectedly.

Result: The inaccuracy is caused by exponent alignment and IEEE 754 rounding at limited precision. Double precision plus stable summation preserves the small contribution and improves the numerical accuracy.

Code of Conduct:

I Athisaya U / 192571001 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:


MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided